AAROGYA DRISHTI

Complete Project Concept and Functional Requirements

Project theme: AI-assisted oral health screening, oral cancer risk awareness, and connection to qualified healthcare professionals.

Purpose of this document: This is a project brief for students. Share it with an AI assistant to generate a detailed software requirements document, system architecture, database design, development phases, and implementation prompts for tools such as GitHub Copilot.

1. Project Overview

Aarogya Drishti is a digital healthcare platform focused on oral health and oral cancer awareness. It is intended to help people complete a preliminary oral health screening, understand possible risk indicators, and connect with registered hospitals and qualified oral healthcare professionals.

Patients will answer questions about symptoms and lifestyle risk factors, upload a photograph of an affected area inside the mouth, and receive an AI-assisted preliminary screening report. The application will use the Google Gemini API through a secure backend integration to describe visible image observations and combine them with the patient's questionnaire responses.

Based on the screening information, patients can search for nearby registered hospitals and doctors, request normal or urgent appointments, and consult a doctor through a secure video consultation when available.

The platform has four roles: Admin, Hospital Manager, Doctor, and Patient. Each role must have a separate dashboard and role-specific permissions, while all modules operate as one integrated application using a shared backend and database.

2. User Roles and Responsibilities

2.1 Admin – Platform Administrator

The Admin has platform-wide administrative control. The Admin creates hospitals and their initial Hospital Manager accounts. Hospital managers must not be able to create new hospitals.

- Create, view, update, activate, and deactivate hospital records.
- Generate a unique Hospital ID and initial password for each hospital manager and securely provide the credentials to the hospital.
- Maintain hospital name, registration information, address, geographical location, contact details, facilities, and services.
- View platform-wide totals for hospitals, hospital managers, doctors, patients, screenings, appointments, and consultations.
- Monitor screening activity and cases marked for prompt or urgent clinical evaluation.
- View appointment and consultation activity across the platform, subject to appropriate privacy and access controls.
- Manage platform users, roles, permissions, and platform-level settings.
- Review audit logs and important administrative activities.

Access rule: Admin has platform-level access. All access to sensitive patient information must be justified, permission-controlled, and audit-logged.

2.2 Hospital Manager – Hospital Operations

Each hospital receives a unique Hospital ID and login credentials created by the Admin. A Hospital Manager can manage only the hospital assigned to that account.

- **Hospital profile:** View and update permitted hospital details, registration information, address, map location, contact details, facilities, services, and working hours.
- **Doctor management:** Add, view, update, and manage doctors who work at that hospital.
- **Doctor information:** Collect full name, photograph, phone number, email, medical qualifications, registration number, specialization, years of experience, previous hospitals or clinics, previous work details, professional achievements, areas of expertise, consultation fees, availability, and working status.
- **Document verification:** Upload and review relevant qualification certificates, medical registration proof, experience documents, and other professional evidence before enabling a doctor account.
- **Doctor account:** Generate a unique Doctor ID and initial password, provide credentials securely, and require the doctor to change the initial password.
- **Hospital overview:** View the hospital's doctor count, patient count, appointments, upcoming appointments, completed consultations, pending requests, and emergency requests.
- **Availability:** View doctor schedules and manage hospital-level operational information.

Access rule: A Hospital Manager must never view or change another hospital's doctors, patients, appointments, or records. Patient medical information should be visible only where the manager has a defined operational need and explicit permission.

2.3 Doctor – Clinical Consultation

Doctors log in using the Doctor ID and password created by their Hospital Manager. Each doctor has an individual dashboard and can access only the patients and records they are authorized to treat.

- View and manage permitted professional profile details, qualifications, specialization, experience, and consultation availability.
- Set available consultation days, time slots, and consultation fees where applicable.
- View assigned patients and patients who have booked or been referred for consultation.
- Review relevant patient information, questionnaire responses, uploaded oral images, AI screening observations, risk category, and previous authorized screening records.
- Review AI-generated information critically; do not treat it as a confirmed diagnosis.
- Accept, reject, or manage appointment requests and view upcoming consultations.
- Conduct in-person or secure video consultations.
- Add consultation notes, clinical observations, recommendations, referrals, and follow-up instructions.
- Recommend an in-person examination, diagnostic tests, biopsy, or specialist referral when clinically appropriate.
- Maintain consultation history and follow-up records.

Access rule: Doctors should access only records necessary for their care relationship with the patient. Access should be recorded in audit logs.

2.4 Patient – Screening and Healthcare Access

Patients use a simple interface designed for the general public. Patient login should use a mobile number and OTP rather than a conventional username and password.

1. Enter a mobile number.
2. Receive an OTP through the configured SMS/OTP provider.

3. Verify the OTP.
4. Create or access the patient's personal profile and dashboard.

Each patient must have a unique account and a private history containing their own screening reports, images, questionnaire responses, appointments, consultations, and follow-up information.

- View and update permitted personal profile details.
- Start a new oral screening and complete the questionnaire.
- Capture or upload an image of the affected oral area.
- View current and previous AI-assisted screening reports.
- Find nearby registered hospitals and doctors.
- View doctor qualifications, specialization, experience, hospital, fees, and availability.
- Book normal or urgent appointments.
- Join an accepted video consultation.
- View consultation notes and follow-up recommendations made available to them.
- Download or share reports securely where permitted.

3. Patient Oral Health Screening

3.1 Start a Screening

After logging in, the patient selects 'Start Oral Screening'. The application explains the purpose of the screening, obtains appropriate consent, and collects the information required for a preliminary assessment.

3.2 Patient Questionnaire

Personal and health information:

- Age and gender, where relevant to the screening workflow.
- General location, if the patient chooses to provide it for nearby hospital discovery.
- Previous oral health problems or relevant medical history.

Lifestyle and risk-factor questions:

- Do you use tobacco? Yes / No.
- If yes, what type: cigarettes, cigars, chewing tobacco, gutka, khaini, pan masala containing tobacco, or other forms.
- Do you consume alcohol? Yes / No / Occasionally.
- Do you chew betel nut or areca nut? Yes / No.
- Where relevant, collect frequency and duration of these habits.

Symptoms checklist:

- White patches inside the mouth.
- Red patches inside the mouth.
- Mouth ulcers or sores that do not heal.
- Unusual growths, lumps, or thickened areas.
- Difficulty opening the mouth.
- Difficulty swallowing.
- Pain while chewing.

- Bleeding inside the mouth.
- Persistent mouth pain or numbness.
- Changes in voice or a lump/swelling in the neck.

Symptom duration: Less than one week; 1–2 weeks; 2–4 weeks; 1–3 months; or more than 3 months. Allow the patient to describe other symptoms in their own words.

The questionnaire should be configurable and should not force a patient to answer questions they do not know. Explain why sensitive information is being collected and obtain appropriate consent.

3.3 Oral Image Capture and Upload

The patient should be guided to capture a clear photograph of the affected area. Provide simple instructions about lighting, focus, positioning, and avoiding blurry images.

The application should support relevant oral regions such as the tongue, inner cheeks, gums, lips, floor of the mouth, roof of the mouth, and other affected areas. The patient should be able to identify the region photographed and upload more than one image if the workflow allows.

The system should validate file type and size, show an image preview, allow replacement or removal before submission, and securely store the image. If the image is unclear or unsuitable, request a new image instead of presenting an unreliable result.

3.4 Gemini API – AI-Assisted Image Analysis

The backend sends the uploaded image and relevant questionnaire information to the Google Gemini API. The API key must remain on the server and must never be exposed in frontend code.

The AI workflow may identify or describe visible features such as white areas, red areas, ulcers, suspicious-looking lesions, unusual tissue changes, visible growths, or image-quality problems. It should return a structured result that can be stored and displayed consistently.

Suggested screening report fields:

- Image quality: adequate, inadequate, or uncertain.
- Observed visible features, described neutrally.
- Reported oral region.
- Preliminary risk category and explanation.
- Patient-reported symptoms and their duration.
- Relevant lifestyle risk factors reported by the patient.
- Recommended next steps and urgency of professional assessment.
- A clear statement of uncertainty and limitations.

Suggested categories: Low Risk; Moderate Risk; High Risk; Urgent Clinical Evaluation Recommended. These are triage or follow-up categories, not cancer stages.

The system must not claim that an image proves cancer, must not assign a definitive cancer stage from a photograph, and must not replace clinical examination. Cancer diagnosis and staging require qualified clinical assessment and appropriate investigations, including biopsy when indicated. AI output must be clinically validated before real-world use.

3.5 Display the Screening Result

Present the result in simple language. Show the preliminary category, image observations, relevant answers, limitations, and recommended next steps. Provide clear actions such as 'Find Nearby Doctors' and 'Book a Consultation'.

If the result indicates high risk or urgent clinical evaluation, clearly recommend prompt professional assessment. If the patient reports severe or rapidly worsening symptoms, direct them to appropriate emergency medical services. Do not make urgent care dependent only on an AI score.

4. Nearby Hospital and Doctor Discovery

After screening, patients should be able to find hospitals and doctors registered on the platform. Use the patient's device location only after permission is granted; also allow manual location or area search.

- Display registered nearby hospitals on a map and in a list.
- Show hospital name, address, contact details, services, and directions.
- Show doctors associated with each hospital.
- Display verified qualifications, registration status, specialization, years of experience, expertise, consultation fee, and availability.
- Allow filtering by location, hospital, specialization, experience, appointment availability, and online consultation availability.
- Show only hospitals and doctors created and managed through the platform's authorized workflows.
- Clearly distinguish verified details from information that has not yet been verified.

The platform should not imply that a doctor is available for an immediate consultation unless the doctor's current availability has been confirmed.

5. Appointment Booking

5.1 Normal Appointment

1. Patient selects a hospital.
2. Patient selects a doctor.
3. Application displays that doctor's available dates and time slots.
4. Patient chooses a slot and submits the appointment request.
5. System checks that the slot is still available and prevents double booking.
6. Doctor receives the request and accepts or declines it.
7. Patient receives the appointment status and confirmation.
8. The appointment appears in the patient and doctor dashboards.

Appointment records should include patient, doctor, hospital, date, time, appointment type, reason or screening reference, status, and relevant notes.

5.2 Urgent or Emergency Consultation Request

Provide a clearly labelled urgent consultation request for patients who need prompt attention. The system can notify available doctors and show the response status.

- Allow the patient to request urgent attention and provide a brief reason.
- Notify suitable available doctors or the relevant hospital.
- Allow a doctor or authorized hospital staff member to accept or respond.
- Show whether the request is pending, accepted, declined, or redirected.
- Offer available video consultation or hospital visit options where appropriate.
- Show hospital contact details and directions.

The application must not promise immediate medical attention unless a qualified provider has accepted the request. It must clearly instruct patients with severe symptoms to contact local emergency services or go to an appropriate emergency department.

6. Secure Video Consultation

The platform should support live video consultations between patients and doctors when a consultation is accepted and scheduled or otherwise authorized.

- Appointment-linked access to the video consultation.
- Live video and audio communication.
- Camera and microphone controls.
- Join and end call controls.
- Display the identities and roles of participants.
- Doctor consultation notes and recommendations.
- Follow-up instructions and next appointment details.
- A record of consultation date, duration, participants, and status.

Use a suitable secure provider or WebRTC-based implementation. Protect call access with authenticated, time-limited permissions. Do not record calls by default; if recording is ever introduced, obtain explicit informed consent and apply appropriate legal and security controls.

7. Patient Screening and Medical History

Every patient's records must remain associated with that patient's account. For example, if a patient completes several screenings over six months, all reports should remain available in chronological order.

- Screening dates and report identifiers.
- Uploaded images and the oral region selected.
- Questionnaire responses and reported symptoms.
- AI observations, preliminary category, and recommendations.
- Doctor consultation details and notes made available to the patient.
- Appointment history and status.
- Clinical reports or documents uploaded by the patient or authorized healthcare professionals.
- Follow-up recommendations and dates.

Patients should be able to view their own history. Doctors should see relevant history only when authorized for that patient's care. A hospital manager should not automatically receive unrestricted access to patient medical records.

A comparison feature may help doctors review changes in submitted images and symptoms over time. It must not describe visual differences as proof of cancer progression or improvement.

8. Dashboards

8.1 Admin Dashboard

- Total hospitals, hospital managers, doctors, and patients.
- Total screenings and screening categories.
- High-risk and urgent-evaluation cases requiring follow-up visibility.

- Total, upcoming, pending, and completed appointments.
- Hospital registrations and recent platform activity.
- Hospital account status and administrative controls.

8.2 Hospital Manager Dashboard

- Hospital profile and status.
- Total doctors and doctor account status.
- Patient and appointment counts for the hospital, subject to privacy permissions.
- Upcoming and pending appointments.
- Completed consultations and urgent requests.
- Doctor schedules and availability.

8.3 Doctor Dashboard

- Doctor profile and availability.
- Assigned or authorized patient list.
- Upcoming, pending, accepted, and completed appointments.
- Patient screening reports and relevant history.
- Video consultation access.
- Consultation notes, referrals, and follow-up records.

8.4 Patient Dashboard

- Personal profile.
- Start new oral screening.
- Current and previous screening reports.
- Screening and consultation history.
- Nearby hospitals and doctors.
- Normal and urgent appointment requests.
- Upcoming appointments and video consultation access.
- Follow-up recommendations and report downloads.

9. Notifications

Provide notifications through suitable channels such as in-app notifications, SMS, email, or push notifications, depending on user consent and available integrations.

- OTP delivery and login alerts.
- Appointment request, acceptance, rejection, cancellation, and reminder.
- Doctor availability or consultation updates.
- Urgent consultation request and response.
- New report availability.
- Follow-up reminders.
- Hospital or doctor account notifications.

10. Security, Privacy, and Access Control

Aarogya Drishti handles sensitive health information. Security and privacy must be designed into every module, not added at the end.

- Implement role-based access control for Admin, Hospital Manager, Doctor, and Patient.
- Use OTP verification for patients and secure password hashing for staff accounts.
- Use secure sessions or tokens, appropriate token expiry, logout, and account recovery.
- Enforce hospital-level data separation on the backend, not only in the frontend.
- Ensure patients can access only their own records.
- Ensure doctors access only records required for authorized care.
- Protect uploaded images, reports, and professional documents using private storage and controlled access.
- Encrypt data in transit and apply appropriate encryption and key-management practices at rest.
- Validate and sanitize all inputs and uploaded files.
- Use rate limiting, secure headers, access logging, and protection against common web attacks.
- Keep API keys and secrets in server-side environment variables or a secrets manager.
- Maintain audit logs for sensitive record access and important administrative actions.
- Obtain informed consent for data collection, image processing, and relevant sharing.
- Define data retention, deletion, backup, and breach-response processes in line with applicable law.

11. Suggested Technology and Integrations

The exact stack should be aligned with the application already built. The following are suggested options, not a requirement to replace existing technologies.

- **Frontend:** React.js.
- **Styling:** CSS, Tailwind CSS, or the existing design system.
- **Backend:** Node.js and Express.js.
- **Database:** MongoDB.
- **AI-assisted image analysis:** Google Gemini API, integrated through the backend.
- **Authentication:** OTP provider for patients and secure ID/password authentication for hospital staff.
- **Image and document storage:** Private Cloudinary configuration or secure object storage.
- **Video consultation:** WebRTC or a suitable secure video communication provider.
- **Maps and location:** Google Maps Platform or another suitable mapping service.
- **Notifications:** SMS, email, and push notification services as required.

Do not rebuild the application from scratch without first reviewing the existing codebase, database, routes, components, and integrations. Preserve working functionality and extend the current architecture where appropriate.

12. Suggested Data Entities

Students should ask for a detailed database schema and relationships based on the existing project. At a high level, the system may require the following entities or collections:

- Users and role assignments.
- Hospitals.

- Hospital manager accounts.
- Doctor profiles and professional documents.
- Patient profiles and consent records.
- Screening sessions.
- Questionnaire responses and symptoms.
- Uploaded oral images and storage metadata.
- AI screening reports and model/API metadata.
- Appointments and time slots.
- Video consultation sessions.
- Doctor consultation notes and recommendations.
- Follow-up records and notifications.
- Audit logs.

Define references, ownership, access rules, indexes, validation, retention, and deletion behaviour for each entity. Avoid duplicating sensitive patient information unnecessarily.

13. Functional and Quality Requirements

- All four roles must use one integrated application, backend, and database.
- Every role must see only the pages and actions permitted for that role.
- Hospital managers can manage only their own hospital.
- Doctors must be linked to a registered hospital and verified according to the platform's process.
- Patients must have unique accounts and private medical histories.
- Screening reports, questionnaire answers, and images must be saved and retrievable according to permissions.
- Appointments must be linked to the relevant patient, doctor, and hospital.
- Time slots must be checked on the backend to prevent double booking.
- All forms must have frontend and backend validation.
- All actions must show appropriate loading, success, empty, and error states.
- The application must be responsive on mobile, tablet, and desktop.
- Use accessible labels, readable text, clear instructions, and simple navigation.
- Do not leave important buttons, forms, filters, or dashboard actions non-functional.
- Use realistic test data during development, but do not present dummy data as live patient or hospital records.
- Test permissions and security, including attempts to access another hospital's or patient's records.

14. Suggested Development Phases

1. **Phase 1 – Existing Application Review:** Inspect the current frontend, backend, database, authentication, folder structure, and working features. Identify what is complete, incomplete, or needs correction.
2. **Phase 2 – Requirements and Architecture:** Prepare the software requirements specification, user flows, role-permission matrix, system architecture, API plan, and database relationship design.
3. **Phase 3 – Authentication and Roles:** Implement Admin, Hospital Manager, Doctor, and Patient authentication, role-based routing, backend authorization, and account lifecycle.

4. **Phase 4 – Admin and Hospital Management:** Implement hospital creation, manager accounts, hospital profiles, and hospital-level data separation.
5. **Phase 5 – Doctor Management:** Implement doctor profiles, professional document uploads, verification workflow, doctor credentials, and availability.
6. **Phase 6 – Patient Registration and Screening:** Implement patient OTP login, profile, consent, questionnaire, image capture/upload, and screening history.
7. **Phase 7 – Gemini Integration:** Implement secure backend integration, image-quality handling, structured AI output, report storage, failure handling, and safety messaging.
8. **Phase 8 – Hospital and Doctor Discovery:** Implement location permission, manual search, map/list views, doctor profiles, and filters.
9. **Phase 9 – Appointments and Notifications:** Implement time slots, booking, acceptance, cancellation, urgent requests, and notifications.
10. **Phase 10 – Video Consultation:** Implement secure appointment-linked video calls and consultation records.
11. **Phase 11 – History and Reports:** Implement patient history, doctor review, report viewing, PDF export, and follow-up management.
12. **Phase 12 – Testing and Deployment:** Conduct functional, integration, security, privacy, usability, and clinical validation; fix defects; prepare deployment and monitoring.

15. Rules Students Must Follow

1. Do not build four disconnected applications. Build one integrated platform with four role-specific dashboards.
2. Do not create a second backend or database if the existing application already has a suitable one.
3. Review the existing code before modifying it. Preserve working features.
4. Admin alone creates hospitals and initial Hospital Manager accounts.
5. Hospital Managers can manage only their own hospital and its doctors.
6. Doctors must be linked to a hospital and have appropriate professional verification.
7. Patients log in using mobile number and OTP.
8. Patients must have separate accounts and private histories.
9. AI screening results and their source images must be stored securely and linked to the correct patient and screening session.
10. Every appointment must reference the correct patient, doctor, and hospital.
11. Enforce all permissions in backend APIs and database queries; hiding a frontend button is not security.
12. Never expose Gemini API keys, OTP secrets, passwords, or private storage credentials in frontend code.
13. Use reusable components, clear folder structure, meaningful names, and consistent error handling.
14. Test normal flows, invalid inputs, expired OTPs, unavailable doctors, duplicate bookings, failed AI requests, and unauthorized access.
15. Do not claim that AI can diagnose or stage oral cancer from a photograph.

16. Expected Project Deliverables

Before implementation, students should prepare:

- A complete Software Requirements Specification (SRS).
- Role-wise functional requirements and permission matrix.

- System architecture and workflow diagrams.
- Database schema, relationships, and indexes.
- Frontend page and dashboard list.
- Backend API endpoint documentation.
- Authentication and authorization design.
- Gemini API integration plan and structured response format.
- Image storage, privacy, and consent plan.
- Appointment and video consultation workflow.
- Development phases with task breakdown.
- Testing plan and deployment plan.

After implementation, students should provide the working integrated application, source code, environment-variable example file without secrets, API documentation, database documentation, test results, and setup instructions.

17. Final Project Objective

The ultimate goal of Aarogya Drishti is to create an integrated digital healthcare ecosystem focused on oral health awareness, AI-assisted preliminary screening, and access to qualified healthcare professionals.

The platform connects four essential groups:

- **Admin:** Creates and manages hospitals and controls the overall platform.
- **Hospital Manager:** Manages the hospital, doctors, and hospital operations.
- **Doctor:** Reviews authorized patient information, conducts consultations, and recommends clinical care.
- **Patient:** Completes screening, views reports, finds nearby healthcare professionals, books appointments, and accesses consultations.

The patient journey should be simple: register using OTP, answer the screening questions, upload an oral image, receive a preliminary AI-assisted report, find a suitable hospital or doctor, book a consultation, and keep all records in one secure patient account.

Vision: Early awareness. Smarter screening. Better access to care.

18. Instructions for Using This Brief with AI Coding Tools

Students should not paste the entire project brief into a coding tool and ask it to generate the whole application at once. First ask an AI assistant to convert this brief into a detailed project documentation package and a phased implementation plan.

Then ask it to generate small, specific implementation prompts in the correct order. Give one prompt at a time to GitHub Copilot, review the changes, run the application, test the feature, and only then proceed to the next phase.

The AI assistant should be instructed to inspect the existing codebase before proposing changes, avoid overwriting working functionality, identify assumptions, ask questions when requirements are unclear, and never invent completed integrations.

Suggested instruction to give an AI assistant:

“Using the Aarogya Drishti project brief above, prepare a professional Software Requirements Specification (SRS), complete system architecture, role and permission matrix, user flow diagrams, database schema, API endpoint documentation, frontend module breakdown, security and privacy requirements, testing strategy, and phased development roadmap. After preparing the documentation, generate detailed, sequential GitHub Copilot implementation prompts for each phase. Assume an existing application may already be available: require the

developer to inspect the current codebase first, preserve working features, and propose changes that fit the existing architecture. Do not generate the entire application in one step. Clearly identify assumptions and unresolved questions."

End of Project Brief